

Structures de données et complexité des algorithmes

**Manipulation des données et analyse de la performance
algorithmique**

Pr. Khadija SLIMANI

khadija.slimani@esiea.fr

PASS Java — Année universitaire 2026–2027

Plan du cours

Séance 3 — Structures de données simples

1. Tableaux : définition, syntaxe, parcours
2. Opérations courantes sur les tableaux
3. Intérêt des tableaux — quand les utiliser ?
4. Chaînes de caractères
5. Chaînes vs tableaux

Séance 4 — Complexité des algos, recherche

6. Complexité des algorithmes (notation grand O)
7. Recherche séquentielle
8. Recherche dichotomique
9. Tri : Quicksort
10. Tri : Mergesort

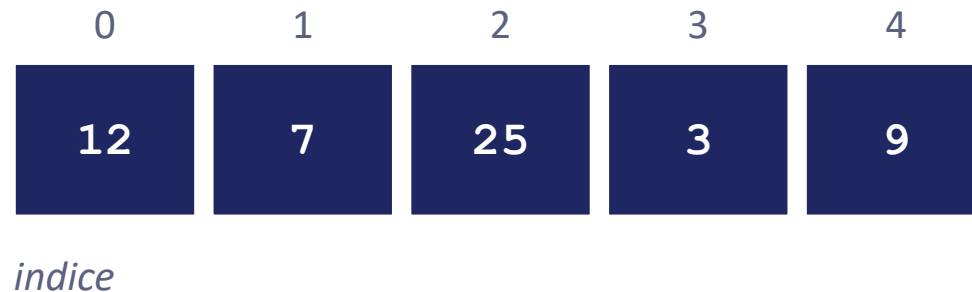
Séance 3

Structures de données simples

Tableaux, chaînes de caractères — représentation, opérations et bon usage.

Qu'est-ce qu'un tableau ?

Un tableau est une structure de données qui regroupe plusieurs valeurs de même type, stockées de manière contiguë et accessibles directement par un indice numérique (souvent à partir de 0).



- **Homogène** tous les éléments sont du même type.
- **Taille définie** le nombre d'éléments est fixé à la déclaration.
- **Accès direct** $T[i]$ renvoie l'élément en position i , en temps constant.

Déclaration et syntaxe en pseudocode

PSEUDOCODE — déclaration

Tableau T[5] de type Entier

T[0] ← 12

T[1] ← 7

T[2] ← 25

T[3] ← 3

T[4] ← 9

- **Tableau T[taille]** réserve un emplacement pour "taille" éléments.
- **de type Entier** précise le type des éléments (Entier, Réel, Caractère...).
- **T[i] ← valeur** affecte une valeur à la position i (← = affectation).
- **Indices** vont de 0 à taille-1 dans la plupart des conventions.

Parcourir et accéder aux éléments

PSEUDOCODE — parcours

```
Pour i de 0 à taille-1 faire  
    Afficher T[i]  
FinPour
```

Ce type de boucle parcourt chaque élément une seule fois : c'est la base de la plupart des traitements sur les tableaux (affichage, somme, recherche...).

- **Accès direct** $T[i]$ est utilisable immédiatement, sans parcourir les autres éléments.
- **Parcours complet** nécessaire quand on ne connaît pas la position recherchée.
- **Compteur i** sert à la fois d'indice et de condition d'arrêt de boucle.

Opérations courantes

R

Lecture

$T[i]$ renvoie la valeur en position i .

Coût : $O(1)$ → le temps d'accès ne dépend pas de la taille du tableau

M

Modification

$T[i] \leftarrow v$ remplace la valeur en position i . Coût : $O(1)$.

?

Recherche

parcourir le tableau pour trouver une valeur. Coût : $O(n)$ au pire.

+/-

Insertion / suppression

décale souvent les éléments suivants. Coût : $O(n)$.

Calculer la somme et la moyenne d'un tableau

PSEUDOCODE

```
Tableau T[5] ← [4, 8, 6, 2, 10]
somme ← 0
Pour i de 0 à 4 faire
    somme ← somme + T[i]
FinPour
moyenne ← somme / 5
```

i	T[i]	somme (cumulée)
0	4	4
1	8	12
2	6	18
3	2	20
4	10	30

Résultat : somme = 30, moyenne = $30 / 5 = 6$

« somme » est une variable qui « accumule » au fil de la boucle — elle garde la trace du total au fur et à mesure.

Avantages, limites et bon usage

Avantages

- Accès très rapide par indice ($O(1)$).
- Structure simple et compacte en mémoire
- Idéal pour des collections homogènes.

Limites

- Taille fixée à l'avance dans de nombreux langages
- Insertion / suppression au milieu coûteuses
- Recherche d'une valeur non triée en $O(n)$



Quel est l'intérêt des tableaux ? Quand les utiliser ?

Dès qu'on connaît le nombre d'éléments à l'avance et qu'on a besoin d'un accès rapide par position, pour des données homogènes et de taille stable.

Qu'est-ce qu'une chaîne de caractères ?

Une chaîne de caractères est une suite ordonnée de caractères. En pseudocode, elle est souvent manipulée comme un cas particulier de tableau : un tableau de caractères.

0	1	2	3
J	A	V	A

`S = "JAVA"`

- **S[i]** accède au caractère en position i, comme pour un tableau.
- **Longueur(S)** renvoie le nombre de caractères de la chaîne.
- **Immuabilité** selon le langage, une chaîne peut être modifiable ou non une fois créée.

Opérations courantes

Lg Longueur(S)

renvoie le nombre de caractères de la chaîne S.

+ Concaténation

S1 + S2 construit une nouvelle chaîne en assemblant S1 et S2.

[] Sous-chaîne

SousChaine(S, début, fin) extrait une portion de la chaîne.

= Comparaison

égalité ou ordre lexicographique (utile pour trier des chaînes).

Compter les voyelles d'un mot

PSEUDOCODE

```
S ← "BONJOUR"
compteur ← 0
Pour i de 0 à Longueur(S)-1 faire
    Si S[i] est une voyelle alors
        compteur ← compteur + 1
    FinSi
FinPour
```

on peut tester « S[i] est une voyelle » avec une condition du type S[i]="A" OU S[i]="E" OU S[i]="I" OU ...

i	S[i]	Voyelle ?	compteur
0	B	non	0
1	O	oui	1
2	N	non	1
3	J	non	1
4	O	oui	2
5	U	oui	3
6	R	non	3

Résultat : 3 voyelles trouvées dans "BONJOUR"

Chaînes de caractères vs tableaux

Aspect	Tableau	Chaîne de caractères
Contenu	Tout type de données	Caractères uniquement
Accès à un élément	$T[i] — O(1)$	$S[i] — O(1)$
Modification	Directe (selon le langage)	Souvent immuable (selon le langage)
Opérations clés	Parcours, tri, recherche	Concaténation, sous-chaîne, comparaison

Ce qu'il faut retenir

- Un tableau regroupe des valeurs homogènes, accessibles en $O(1)$ via un indice.
- Le tableau est adapté quand la taille des données est connue ou stable.
- Lecture et modification sont rapides ; insertion et suppression au milieu coûtent $O(n)$.
- Une chaîne de caractères est un cas particulier de tableau : un tableau de caractères.

Séance 4

Complexité des algorithmes & recherche

Complexité, recherche séquentielle et dichotomique, Quicksort et Mergesort.

Qu'est-ce que la complexité algorithmique ?

La complexité algorithmique mesure comment le temps d'exécution (complexité temporelle) ou la mémoire utilisée (complexité spatiale) d'un algorithme évolue en fonction de la taille n des données en entrée.

Notation grand O

Décrit le comportement asymptotique d'un algorithme quand n devient grand, en ignorant les constantes et les termes non dominants. Elle donne une borne sur le pire cas.

- **Complexité temporelle** nombre d'opérations effectuées en fonction de n .
- **Complexité spatiale** quantité de mémoire supplémentaire utilisée.

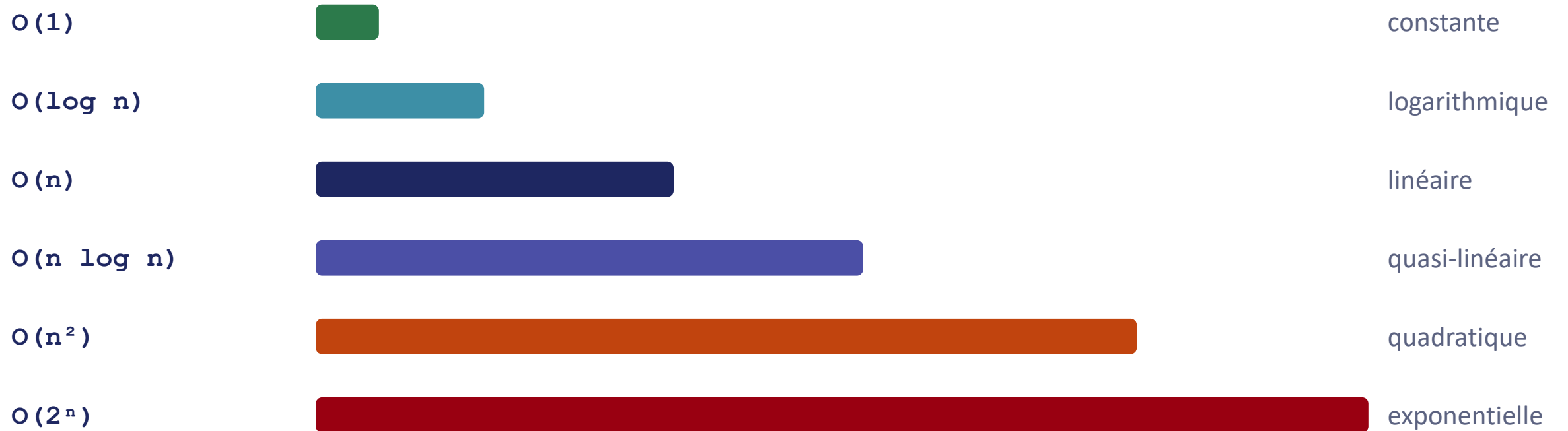
Pourquoi mesurer la complexité ?

Le choix d'un algorithme peut changer radicalement le nombre d'opérations nécessaires, surtout quand n devient grand.

n (taille des données)	Algorithme $O(n)$	Algorithme $O(n^2)$
1 000	~1 000 opérations	~1 000 000 opérations
1 000 000	~1 000 000 opérations	~1 000 000 000 000 opérations

À un million d'éléments, un algorithme quadratique devient impraticable alors qu'un algorithme linéaire reste utilisable — d'où l'importance de choisir le bon algorithme.

Les classes de complexité courantes



Du plus rapide (haut) au plus lent (bas) — longueur de la barre = croissance relative illustrative, pas une échelle exacte.

Meilleur cas, cas moyen, pire cas



Meilleur cas

le scénario le plus favorable pour l'algorithme (borne inférieure).



Cas moyen

le comportement "typique", sur des données aléatoires.



Pire cas

le scénario le plus défavorable — généralement retenu pour garantir une borne.

En pratique, on utilise le plus souvent la complexité du pire cas : elle garantit que l'algorithme ne dépassera jamais cette borne, quelles que soient les données.

Recherche séquentielle : principe

On parcourt le tableau élément par élément, dans l'ordre, jusqu'à trouver la valeur recherchée ou atteindre la fin du tableau.

0	1	2	3	4	5
8	3	5	1	9	2

Exemple : recherche de la valeur 1 → trouvée en position 3, après avoir examiné 4 éléments.

- **Aucun tri requis** fonctionne sur n'importe quel tableau, trié ou non.
- **Simplicité** algorithme très simple à écrire et à comprendre.

Recherche séquentielle : pseudocode

PSEUDOCODE

```
Fonction RechercheLineaire(T, n, valeur)
    Pour i de 0 à n-1 faire
        Si T[i] = valeur alors
            Retourner i
        FinSi
    FinPour
    Retourner -1
FinFonction
```

Recherche séquentielle : exemple pas à pas

On cherche la valeur 9 dans le tableau $T = [4, 11, 9, 2, 7]$ (non trié).

0	1	2	3	4
4	11	9	2	7

i	T[i]	T[i] = 9 ?
0	4	Non → on continue
1	11	Non → on continue
2	9	Oui → on retourne i = 2

Résultat : trouvé à l'indice 2, en seulement 3 comparaisons.

Recherche séquentielle : complexité

Ω Meilleur cas — $O(1)$

la valeur recherchée est le premier élément.

Θ Cas moyen — $O(n)$

en moyenne, environ $n/2$ comparaisons.

O Pire cas — $O(n)$

la valeur est le dernier élément, ou absente du tableau.

Recherche dichotomique : principe

Sur un tableau trié, on compare la valeur recherchée à l'élément du milieu, puis on élimine la moitié du tableau qui ne peut pas contenir la valeur. On répète sur la moitié restante.

Prérequis indispensable : le tableau doit être trié. Sans tri préalable, la dichotomie ne fonctionne pas.

- **debut / fin** délimitent la zone de recherche courante.
- **milieu** position centrale ; on compare $T[\text{milieu}]$ à la valeur cherchée.
- **À chaque étape** la zone de recherche est divisée par deux.

Recherche dichotomique : pseudocode

PSEUDOCODE

```
Fonction RechercheBinaire(T, n, valeur)
    debut ← 0
    fin ← n - 1
    Tant que debut ≤ fin faire
        milieu ← (debut + fin) / 2
        Si T[milieu] = valeur alors
            Retourner milieu
        Sinon si T[milieu] < valeur alors
            debut ← milieu + 1
        Sinon
            fin ← milieu - 1
    FinSi
FinTantQue
Retourner -1
FinFonction
```

Recherche dichotomique : exemple pas à pas

On cherche la valeur 45 dans le tableau trié $T = [2, 5, 8, 12, 16, 23, 38, 45, 56, 72, 91]$ (indices 0 à 10).

Étape	debut	fin	milieu	T[milieu]	Comparaison
1	0	10	5	23	$23 < 45 \rightarrow \text{debut} = 6$
2	6	10	8	56	$56 > 45 \rightarrow \text{fin} = 7$
3	6	7	6	38	$38 < 45 \rightarrow \text{debut} = 7$
4	7	7	7	45	$45 = 45 \rightarrow \text{trouvé !}$

Résultat : trouvé à l'indice 7, en seulement 4 comparaisons — contre 8 comparaisons pour une recherche séquentielle sur ce même tableau.

Dichotomique vs séquentielle

Critère	Recherche séquentielle	Recherche dichotomique
Tableau trié requis	Non	Oui
Complexité	$O(n)$	$O(\log n)$
Simplicité	Très simple	Un peu plus complexe
Cas d'usage	Petites collections, données non triées	Grandes collections triées

Sur 1 million d'éléments triés :

- *recherche séquentielle → jusqu'à 1 000 000 comparaisons ;*
- *recherche dichotomique → 20 comparaisons au maximum ($\log_2 1\,000\,000 \approx 20$).*

Pourquoi trier des données ?

- **Faciliter la recherche** : un tableau trié permet d'utiliser la recherche dichotomique en $O(\log n)$.
- **Lisibilité** : présenter des données de façon ordonnée pour l'utilisateur ou l'analyse.
- **Prérequis d'autres algorithms** : de nombreux traitements (fusion, détection de doublons...) supposent des données triées.

Deux stratégies « diviser pour régner » très utilisées : Quicksort et Mergesort.

Quicksort : principe

Quicksort est un algorithme « diviser pour régner » : on choisit un pivot, on partitionne le tableau en éléments plus petits et plus grands que le pivot, puis on trie récursivement chaque partie.

1 Choisir un pivot

souvent le dernier (ou premier) élément du sous-tableau.

2 Partitionner

regrouper les éléments $\leq$ pivot à gauche, $>$ pivot à droite.

3 Récursion

appliquer Quicksort sur chacune des deux parties.

Quicksort : pseudocode

PSEUDOCODE — tri

```
Fonction Quicksort(T, debut, fin)
    Si debut < fin alors
        pivot ← Partitionner(T, debut, fin)
        Quicksort(T, debut, pivot - 1)
        Quicksort(T, pivot + 1, fin)
    FinSi
FinFonction
```

PSEUDOCODE — partition

```
Fonction Partitionner(T, debut, fin)
    pivot ← T[fin]
    i ← debut - 1
    Pour j de debut à fin-1 faire
        Si T[j] ≤ pivot alors
            i ← i + 1
            Echanger(T[i], T[j])
    FinSi
    FinPour
    Echanger(T[i+1], T[fin])
    Retourner i + 1
FinFonction
```

Quicksort : exemple pas à pas

Départ [8, 3, 5, 1, 9, 2]

Pivot = dernier élément (2).



Étape 1 [1, 2, 5, 8, 9, 3]

Partition autour de 2 : les éléments ≤ 2 passent à gauche.



Étape 2 [1, 2, 3, 8, 9, 5]

Partition du sous-tableau [5, 8, 9, 3] autour de 3.



Étape 3 [1, 2, 3, 5, 9, 8]

Partition du sous-tableau [8, 9, 5] autour de 5.



Résultat [1, 2, 3, 5, 8, 9]

Tableau trié.

Quicksort : complexité

Θ Cas moyen — $O(n \log n)$

avec un bon choix de pivot, les partitions sont équilibrées.

O Pire cas — $O(n^2)$

tableau déjà trié avec un mauvais choix de pivot (partitions déséquilibrées).

M Mémoire

tri en place : peu de mémoire supplémentaire (pile de récursion).

Quicksort n'est pas stable : deux éléments égaux peuvent voir leur ordre relatif changer après le tri.

Mergesort : principe

Mergesort est aussi un algorithme « diviser pour régner » : on divise le tableau en deux moitiés, on trie récursivement chaque moitié, puis on fusionne les deux moitiés triées.

1 Diviser

couper le tableau en deux moitiés,
jusqu'à obtenir des éléments seuls.

2 Régner

trier récursivement chaque moitié
(trivial pour un seul élément).

3 Fusionner

combiner deux moitiés triées en
une seule liste triée.

Mergesort : pseudocode

PSEUDOCODE — tri

```
Fonction Mergesort(T, debut, fin)
    Si debut < fin alors
        milieu ← (debut+fin) / 2
        Mergesort(T, debut, milieu)
        Mergesort(T, milieu+1, fin)
        Fusionner(T, debut, milieu, fin)
    FinSi
FinFonction
```

PSEUDOCODE — fusion

```
Fonction Fusionner(T, debut, milieu, fin)
    Créer Gauche ← T[debut..milieu]
    Créer Droite ← T[milieu+1..fin]
    i, j ← 0, 0
    Pour k de debut à fin faire
        Choisir le plus petit de
        Gauche[i] et Droite[j],
        le placer dans T[k],
        avancer i ou j
    FinPour
FinFonction
```

Mergesort : exemple pas à pas

Départ [8, 3, 5, 1, 9, 2]

 ▼

Diviser [8, 3, 5] [1, 9, 2]

 ▼

Diviser [8, 3] [5] [1, 9] [2]

 ▼

Diviser [8] [3] [5] [1] [9] [2]

Éléments seuls : déjà « triés ».

Fusionner [3, 8] [5] [1, 9] [2]

 ▼

Fusionner [3, 5, 8] [1, 2, 9]

 ▼

Résultat [1, 2, 3, 5, 8, 9]

Tableau trié.

Mergesort : complexité

Θ Tous les cas — $O(n \log n)$

meilleur, moyen et pire cas ont la même complexité.

S Stable

l'ordre relatif des éléments égaux est conservé.

M Mémoire — $O(n)$

nécessite un espace supplémentaire pour la fusion.

La complexité de Mergesort est garantie quel que soit le tableau d'entrée : pas de « pire cas » dégradé comme pour Quicksort.

Quicksort vs Mergesort

Critère	Quicksort	Mergesort
Complexité moyenne	$O(n \log n)$	$O(n \log n)$
Complexité pire cas	$O(n^2)$	$O(n \log n)$
Mémoire supplémentaire	$O(\log n)$ — pile de récursion	$O(n)$
Stable	Non	Oui
En pratique	Souvent plus rapide (petites constantes)	Complexité garantie, adapté au tri externe

Ce qu'il faut retenir

- La complexité (notation grand O) permet de comparer des algorithmes indépendamment du matériel.
- Quicksort et Mergesort sont deux algorithmes « diviser pour régner », en $O(n \log n)$ en moyenne.
- Recherche séquentielle : $O(n)$, aucun tri requis.
- Recherche dichotomique : $O(\log n)$, tableau trié requis.
- Quicksort : pire cas $O(n^2)$, peu de mémoire.
- Mergesort : $O(n \log n)$ garanti, stable, plus de mémoire.

Exercices d'application